COMP4141

Theory of Computation

Lecture 10: (Un)Decidability and Reductions

Outline

Computation problems

So far our computation models have been focused on the question:

Is the word w accepted by this automaton?

Encodings allow us to represent more general computational problems:

Given input I, compute output O

For example:

- On input $\langle I \rangle$, halt with $\langle O \rangle$ on the tape
- Is the word $\langle I, O \rangle$ accepted by the automaton?

Decision Problems

Recall from Lecture 7 the class of **decision problems**: problems for which the answer is either "Yes" or "No".

We can encode such problems as: Is the word $\langle I \rangle$ accepted?

NB

We implicitly restrict ourselves to correct encodings: we generally consider the complement problem *to be the set of inputs I, for which $\langle I \rangle$ is not accepted.*

Terminology: Inputs I are commonly referred to as **instances**. If $\langle I \rangle \in L$ it is a **yes-instance**; otherwise it is a **no-instance**.

Decision Problems: Example

Example

DFA Acceptance

Input: A DFA D and a word w

Question: Does D accept w?

is the decision problem associated with the language:

$$A_{\text{DFA}} = \{ \langle D, w \rangle \mid D \text{ is a DFA that accepts } w \}$$

Decision Problems: Example

Example

A Hamiltonian path in a directed graph is a path that visits every vertex exactly once.

HAMPATH

Input: A graph G

Question: Does G contain a Hamiltonian path?

is the decision problem associated with the language:

$$L_{\mathsf{HP}} = \{ \langle G \rangle \mid G \text{ contains a Hamiltonian path} \}$$

Decision Problems: Example

Example

PRIMES

Input: A number n

Question: Is n prime?

is the decision problem associated with the language:

$$L_{\text{prime}} = \{ \langle n \rangle \mid n \text{ is prime} \}$$

Decision Problems: Example

Example

The complement of PRIMES,

COMPOSITES

Input: A number n

Question: Is n composite?

is the decision problem associated with the language:

$$L_{\text{comp}} = \{ \langle n \rangle \mid n \text{ is composite} \} = \overline{L_{\text{prime}}}$$

NB

This is quite different from factorizing n.

Decision Problems: Example

Example

To formulate factorization as a decision problem, we introduce a *bound*.

FACTORIZATION

Input: A number n and a number k

Question: Does n have a factor smaller than k?

Finding a factorization could then be done using binary search.

Outline

Recursively Enumerable Problems

To show a decision problem is recursively enumerable (or semi-decidable) it is sufficient to come up with a *semi-decision* process:

- For every "Yes" instance, the process accepts the input in finite time.

NB

For "No" instances, the only requirement is that process does not accept.

Recursively Enumerable Example

Theorem

The decision problem:

TM Non-emptiness

Input: *A Turing Machine M*

Question: *Is $L(M) \neq \emptyset$?*

is recursively enumerable.

Recursively Enumerable Example

Theorem

The decision problem:

$$NE_{TM} = \{ \langle M \rangle \mid L(M) \neq \emptyset \}$$

is recursively enumerable.

Recursively Enumerable Example

Theorem

The decision problem:

$$NE_{TM} = \{ \langle M \rangle \mid L(M) \neq \emptyset \}$$

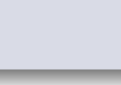
is recursively enumerable.

Proof.

First attempt:

On input M:

- For each $w \in \Sigma^*$ (taken in lenlex order)
  - Simulate M on w
  - If M accepts then ACCEPT



Recursively Enumerable Example

Theorem

The decision problem:

$$NE_{TM} = \{ \langle M \rangle \mid L(M) \neq \emptyset \}$$

is recursively enumerable.

Proof.

Second attempt:

On input M:

- Take an enumeration of Σ^* (e.g. lenlex order)
- For each $i \in \mathbb{N}$:
  - For each word w in the first i words of Σ^*:
    - Simulate M on w for up to i steps
    - If M accepts then ACCEPT

Recursively Enumerable Example

Theorem

The decision problem:

$$NE_{TM} = \{ \langle M \rangle \mid L(M) \neq \emptyset \}$$

is recursively enumerable.

Proof.

Alternative argument:

On input M:

- Non-deterministically guess $w \in \Sigma^*$
- Simulate M on w
- If M accepts then ACCEPT; if M rejects then REJECT

Recursively Enumerable Example

Theorem

The decision problem:

$$NE_{TM} = \{ \langle M \rangle \mid L(M) \neq \emptyset \}$$

is recursively enumerable.

Proof.

Another alternative argument:

On input M:

- Construct an enumerator E that enumerates $L(M)$ [check this is possible!]
- Simulate E
- If E prints a word then ACCEPT

co-Recursively Enumerable

A decision problem is **coRE**, if the complement problem is recursively enumerable.

To show a problem is **coRE** it is sufficient to come up with a process that:

- For every "No" instance, the process rejects the input in finite time

coRE Example

Theorem

The decision problem:

TM Emptiness

Input: *A Turing Machine* M

Question: *Is* $L(M) = \emptyset$*?*

is coRE

Proof.

It is the complement of NE_{TM} □

coRE Example

Theorem

The decision problem:

Divergence

Input: *A Turing Machine M*

Question: *Does M diverge on all inputs?*

is coRE

coRE Example

Theorem

The decision problem:

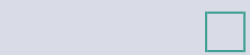
$$DIV = \{ \langle M \rangle \mid M \text{ diverges on all inputs} \}$$

is coRE

Proof.

On input M:

- Run M on all words in Σ^* in a dove-tailing manner.
- If M ever accepts or rejects, then $\textsc{Reject}$



Outline

Decidable Problems

To show a decision problem is decidable (or recursive) it has to be accepted by a decider:

- For every "Yes" instance, the process accepts the input in finite time.
- For every "No" instance, the process rejects the input in finite time.

Decidable Problems for Regular Languages

Recall the acceptance problem for DFAs:

$$A_{\text{DFA}} = \{ \langle D, w \rangle \mid D \text{ is a DFA that accepts } w \}$$

Theorem

A_{DFA} is decidable.

Proof idea.

One TM that decides A_{DFA} would simulate B on w by keeping track of B's current state and of how much of the input word w has been read by writing these data items onto the tape. □

More decidable problems for Regular Languages

Recall from Lecture 7:

$$
\begin{aligned}
A_{\mathrm{NFA}} &= \{\, \langle B, w\rangle \mid B \text{ is an NFA that accepts } w \,\} \\
A_{\mathrm{RE}_\Sigma} &= \{\, \langle R, w\rangle \mid R \in \mathrm{RE}_\Sigma \wedge w \in L(R) \,\} \\
E_{\mathrm{DFA}} &= \{\, \langle B\rangle \mid B \text{ is a DFA and } L(B) = \emptyset \,\} \\
EQ_{\mathrm{DFA}} &= \{\, \langle A, B\rangle \mid A, B \text{ are DFAs and } L(A) = L(B) \,\}
\end{aligned}
$$

are all decidable.

Decidable Problems for CFLs

Also from Lecture 7:

Theorem

The acceptance problem for CFGs:

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ is a CFG that generates } w \}$$

is decidable.

Proof idea.

Convert G to Chomsky NF. Enumerate all derivations with up to $2|w| - 1$ steps (or 1 step if $w = \epsilon$). If any of these derivations generate w $\textsc{Accept}$, else $\textsc{Reject}$. □

Decidable Problems for CFLs

Also from Lecture 7:

Theorem

The acceptance problem for CFGs:

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ is a CFG that generates } w \}$$

is decidable.

Proof idea.

Convert G to Chomsky NF. Enumerate all derivations with up to $2|w| - 1$ steps (or 1 step if $w = \epsilon$). If any of these derivations generate w $\textsc{Accept}$, else $\textsc{Reject}$. □

Alternative proof.

Run the CYK algorithm. □

Outline

Undecidability

Theorem

There are undecidable and there are non-RE languages.

Proof.

The set of TMs is enumerable, hence countable. Each TM accepts (or even decides) a single language. The set of languages is uncountable. □

It is instructive to give examples of non-RE and of RE yet undecidable languages.

An Undecidable Problem

Theorem

The acceptance problem for Turing Machines:

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM that accepts } w \}$$

is RE but undecidable.

Proof idea for RE membership.

Construct a *universal TM* U, which, on input $\langle M, w \rangle$ simulates M on w and ACCEPTs (REJECTs) if M enters q_{accept} (q_{reject}).
U accepts A_{TM} but it doesn't decide it. □

Undecidability proof

Proof idea for undecidability.

Assume to the contrary that H is a decider for A_{TM}. Let D be a TM that uses H as a building block. Its inputs are TM descriptions. D does the following:

On input $\langle M \rangle$:

1. Run H on $\langle M, \langle M \rangle \rangle$
2. If H accepts then $\textsc{Reject}$ else $\textsc{Accept}$

Undecidability proof

Proof idea for undecidability.

Assume to the contrary that H is a decider for A_{TM}. Let D be a TM that uses H as a building block. Its inputs are TM descriptions. D does the following:

On input $\langle M \rangle$:

1. Run H on $\langle M, \langle M \rangle \rangle$
2. If H accepts then $\textsc{Reject}$ else $\textsc{Accept}$

Running D on $\langle D \rangle$ yields:

$$D(\langle D \rangle) = \begin{cases} \textsc{Accept} & \text{if } D \text{ does not accept } \langle D \rangle \\ \textsc{Reject} & \text{if } D \text{ accepts } \langle D \rangle \end{cases}$$

which is a contradiction. □

Non-RE Languages

Recall:

Theorem

$$\text{Rec} = \text{RE} \cap \text{coRE}.$$

Corollary

$\overline{A_{TM}}$ is not RE.

Proof.

by the last 3 theorems. □

Outline

Reductions

Proving undecidability or non-RE from scratch is often cumbersome.

If we can show that some problem B is at least as hard as a known undecidable problem A then we have a proof that B is undecidable, too.

What does "at least as hard as" mean?

It means we can *reduce* every instance of A to an instance of B, that is, we transform the problem of solving A to the problem of solving B. In that way, we can use a solution for B to give a solution for A.

Proofs by Reduction

This is one of the most important proof techniques in computer science.

To prove that problem P_2 is hard, show that there is an "easy" reduction from a known hard problem P_1 to P_2.

Then P_2 is at least as hard as P_1.

Reduction Example

Example

(Suppose it is well-known that Kevin cannot lift a car.)

Theorem

Kevin cannot lift a loaded truck (P_2).

Proof.

By reduction from the car-lifting problem (P_1).

Suppose Kevin could lift a loaded truck.

Then, Kevin could solve the car-lifting problem by putting the car in a truck and lifting the truck.

But, it is known that Kevin cannot lift a car. □

Common pitfall

Important

Make sure you are doing the reduction in the right direction!

known hard problem $\longrightarrow$ new problem

A wrong proof

Example

Theorem

Kevin cannot lift a feather.

Proof.

By reduction to the car-lifting problem.

We can reduce feather lifting to car-lifting by putting the feather in a car.

It is known that Kevin cannot lift a car.

Therefore, Kevin cannot lift a feather! □

A wrong proof

Example

Theorem

Kevin cannot lift a feather.

Proof.

By reduction to the car-lifting problem.

We can reduce feather lifting to car-lifting by putting the feather in a car.

It is known that Kevin cannot lift a car.

Therefore, Kevin cannot lift a feather! □

(How's that again?)

Example: Halting Problem

Theorem

$H_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM that halts on input } w \}$ *is undecidable.*

Proof.

Assume H_{TM} is decidable and then show that a decider D for H_{TM} can be used to build a decider for A_{TM}.

The decider D' for A_{TM} works as follows.

On input $\langle M, w \rangle$:

1. Run D on $\langle M, w \rangle$.
2. If D rejects, then $\textsc{Reject}$
3. Otherwise, run M on w and return what it returns.

This D' decides A_{TM}, however, we know that A_{TM} is undecidable. Consequently, our assumption that D decides H_{TM} must be wrong. □

Example: Halting Problem

Theorem

$H_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM that halts on input } w \}$ *is undecidable.*

Proof.

Assume H_{TM} is decidable and then show that a decider D for H_{TM} can be used to build a decider for A_{TM}.

The decider D' for A_{TM} works as follows.

On input $\langle M, w \rangle$:

1. Run D on $\langle M, w \rangle$.
2. If D rejects, then $\textsc{Reject}$
3. Otherwise, run M on w and return what it returns.

This D' decides A_{TM}, however, we know that A_{TM} is undecidable. Consequently, our assumption that D decides H_{TM} must be wrong. □

This is a proof **by (Turing-) reduction from** A_{TM}.

Many-One Reducibility

Definition

Language A is *many-one reducible* to language B, written $A \leq_{\mathrm{m}} B$, if there is a computable function $f : \Sigma^* \longrightarrow \Sigma^*$, where for every $w \in \Sigma^*$,

$$w \in A \Leftrightarrow f(w) \in B \ .$$

The function f is called the *reduction* from A to B.

Theorem

If $A \leq_m B$ and B is decidable, then A is decidable.

Corollary

If $A \leq_m B$ and A is undecidable, then B is undecidable.

Example: Halting Problem, again

Theorem

$A_{TM} \leq_m H_{TM}$

Example: Halting Problem, again

Theorem

$A_{TM} \leq_m H_{TM}$

Proof.

Given $\langle M, w \rangle$ we construct a TM $H_{\langle M,w \rangle}$ as follows:

$H_{\langle M,w \rangle}$

On input x:

1. Run M on x
2. If M accepts, Accept
3. If M rejects, go into an infinite loop.

Example: Halting Problem, again

Theorem

$A_{TM} \leq_m H_{TM}$

Proof.

Given $\langle M, w \rangle$ we construct a TM $H_{\langle M,w \rangle}$ as follows:

$H_{\langle M,w \rangle}$

On input x:

1. Run M on x
2. If M accepts, Accept
3. If M rejects, go into an infinite loop.

$\langle M, w \rangle \in A_{\mathrm{TM}}$ if and only if $H_{\langle M,w \rangle}$ halts on input w. □

Example: Tutorial problem

Theorem

$$EQ_{RE_\Sigma} \leq_m E_{DFA}$$

Example: Tutorial problem

Theorem

$$EQ_{RE_\Sigma} \leq_m E_{DFA}$$

Proof.

Given E_1, E_2:

- Construct NFAs N_1 and N_2 with $L(N_i) = L(E_i)$
- Construct DFAs D_1 and D_2 with $L(D_i) = L(N_i)$
- Construct DFA D_3 with $L(D_3) = \overline{L(D_2)}$
- Construct DFA D with $L(D) = L(D_1) \cap L(D_3)$
- Output D

Example: Tutorial problem

Theorem

$$EQ_{RE_\Sigma} \leq_m E_{DFA}$$

Proof.

Given E_1, E_2:

- Construct NFAs N_1 and N_2 with $L(N_i) = L(E_i)$
- Construct DFAs D_1 and D_2 with $L(D_i) = L(N_i)$
- Construct DFA D_3 with $L(D_3) = \overline{L(D_2)}$
- Construct DFA D with $L(D) = L(D_1) \cap L(D_3)$
- Output D

$L(E_1) \subseteq L(E_2)$ if and only if $L(D) = \emptyset$ □

Turing Reducibility

Many-one reductions describe a transformation from one problem to another problem.

Turing reductions describe transforming a *solution* to another *solution*: given an algorithm for one problem (the new problem), construct an algorithm for the other one (known hard problem).

Definition

An **oracle** for a language B is an external device that can determine whether a given word w is a member of B. An **oracle (for B) Turing Machine**, M^B, is a Turing machine that has the additional capacity of querying an oracle for B.

Definition

Language A is **Turing reducible** to language B, written $A \leq_T B$ if A can be decided by a Turing Machine with an oracle for B.

Turing Reductions vs Many-One Reductions

The following should be obvious:

Theorem

If $L_1 \leq_m L_2$ *then* $L_1 \leq_T L_2$

The converse does not, in general, hold.

Example

Recall: $\overline{A_{\mathrm{TM}}}$ is not RE.

It follows that $\overline{A_{\mathrm{TM}}} \not\leq_m A_{\mathrm{TM}}$.

However: $\overline{A_{\mathrm{TM}}} \leq_T A_{\mathrm{TM}}$.

Closure Under Reductions

Theorem

RE, coRE, *and* Rec *are all closed under many-one reductions. That is*

$$L_1 \leq_m L_2 \text{ and } L_2 \in \mathrm{RE} \quad \Longrightarrow \quad L_1 \in \mathrm{RE}$$

(and similarly for coRE, *and* Rec*).*

NB

The previous example shows that RE *and* coRE *are not closed under Turing reductions.*

Outline

Rice's Theorem

Definition

Let $P \subseteq \{ \langle M \rangle \mid M \text{ is a TM} \}$ be a language consisting of TM descriptions.

P is *nontrivial* if it contains some but not all TM descriptions.

P is *semantic* if $L(M_1) = L(M_2) \Rightarrow (\langle M_1 \rangle \in P \Leftrightarrow \langle M_2 \rangle \in P)$.

Rice's Theorem

Definition

Let $P \subseteq \{ \langle M \rangle \mid M \text{ is a TM} \}$ be a language consisting of TM descriptions.

P is *nontrivial* if it contains some but not all TM descriptions.

P is *semantic* if $L(M_1) = L(M_2) \Rightarrow (\langle M_1 \rangle \in P \Leftrightarrow \langle M_2 \rangle \in P)$.

Theorem (Rice's Theorem)

All nontrivial semantic properties are undecidable.

Rice's Theorem: Proof

Proof by reduction from A_{TM} .

W.l.o.g. we assume that the TM that always rejects, $T_\emptyset$, (i.e., $L(T_\emptyset) = \emptyset$) is not in P, otherwise we proceed with $\overline{P}$ instead of P. Let T be a TM with $\langle T \rangle \in P$. Given $\langle M, w \rangle$ we construct $N_{\langle M,w \rangle}$ as follows:

$N_{\langle M,w \rangle}$

On input x:

1. Simulate M on w. If it halts and rejects, Reject.
2. Otherwise, simulate T on x. If it accepts, Accept.

We claim that $\langle M, w \rangle \in A_{\mathsf{TM}}$ if, and only if $N_{\langle M,w \rangle} \in P$.

$\Rightarrow$: If $w \in L(M)$ then $N_{\langle M,w \rangle}$ simulates T, so $L(N_{\langle M,w \rangle}) = L(T)$ and $N_{\langle M,w \rangle} \in P$.

$\Leftarrow$: If $w \notin L(M)$ then $N_{\langle M,w \rangle}$ always rejects, so $L(N_{\langle M,w \rangle}) = \emptyset = L(T_\emptyset)$ and so $N_{\langle M,w \rangle} \notin P$.

Applications of Rice's theorem

- Whether a language (of a TM) is empty is undecidable.
- Whether a language (of a TM) is non-empty undecidable.
- Whether a language (of a TM) is regular is undecidable.
- Whether a language (of a TM) is context-free is undecidable.

Applications of Rice's theorem

- Whether a language (of a TM) is empty is undecidable.
- Whether a language (of a TM) is non-empty undecidable.
- Whether a language (of a TM) is regular is undecidable.
- Whether a language (of a TM) is context-free is undecidable.

This does not mean we cannot prove any of this for a given TM. Sometimes we can!

Wrong Applications of Rice's theorem

Rice's theorem cannot be used for these:

- Whether a TM has less than 7 states,
- Whether a TM has a final state,
- Whether a TM has a start state.

(Note how these properties are not properties of languages, but properties of TMs.)

- Whether the language is r.e.
- Whether the language is a subset of Σ^*.

Far-Reaching Consequence

We cannot write a program that reliably answers a non-trivial question about the black-box behaviour of programs (as soon as the programming language used to write the to-be-analysed programs is reasonably expressive).

Far-Reaching Consequence

We cannot write a program that reliably answers a non-trivial question about the black-box behaviour of programs (as soon as the programming language used to write the to-be-analysed programs is reasonably expressive).

We may be able to write programs that answer non-trivial questions about programs themselves.